☸️ Kubernetes — Complete Notes: Beginner to Expert

> Covers every concept for learning, working, and acing interviews.
> Each section has theory, real YAML examples, and interview Q&A.

📋 Table of Contents

1. What is Kubernetes?
2. Kubernetes Architecture
3. Core Objects — Pods
4. ReplicaSet
5. Deployments
6. Services
7. Namespaces
8. ConfigMaps & Secrets
9. Volumes & Persistent Storage
10. StatefulSets
11. DaemonSets
12. Jobs & CronJobs
13. Ingress & Ingress Controllers
14. Resource Management
15. Health Checks — Probes
16. Horizontal & Vertical Pod Autoscaling
17. RBAC — Role-Based Access Control
18. Network Policies
19. Helm — Package Manager
20. Monitoring & Logging
21. Advanced Scheduling
22. Custom Resource Definitions (CRDs)
23. kubectl — Complete Command Reference
24. Top 60 Interview Questions & Answers
25. Quick Reference Cheat Sheet

---

1. What is Kubernetes?

Definition

Kubernetes (K8s) is an **open-source container orchestration platform** originally developed by Google, now maintained by the CNCF (Cloud Native Computing Foundation). It automates deploying, scaling, and managing containerised applications.

Why do we need Kubernetes?

Problem without K8s	How K8s solves it
Containers crash — no auto-restart	Self-healing: restarts failed containers
Traffic spikes — manual scaling	Auto-scaling: HPA/VPA/KEDA
Deploying new versions — downtime	Rolling updates and rollbacks
Many containers — hard to manage	Declarative desired-state management
Container networking — complex	Built-in service discovery and DNS
No resource management	CPU/Memory limits and requests
Secrets in code	Secrets API with encryption

Key Terminology

Term	Meaning
Container	Lightweight, portable application runtime (Docker)
Pod	Smallest deployable unit in K8s — wraps one or more containers
Node	A physical or virtual machine that runs Pods
Cluster	A set of Nodes managed together by K8s
Control Plane	The "brain" of K8s — manages the cluster
Workload	An application running in K8s (Deployment, Job, etc.)
Manifest	A YAML/JSON file describing a K8s object
Desired State	What you tell K8s you want

Term	Meaning
Actual State	What is currently running
Reconciliation	K8s constantly makes actual state = desired state

Kubernetes vs Docker

```
Docker alone:
  You → docker run → Container running on ONE machine

Kubernetes:
  You → kubectl apply → K8s decides WHICH machine, HOW MANY replicas,
        RESTARTS on failure, LOAD BALANCES traffic, SCALES up/down
```

---

2. Kubernetes Architecture

High-Level Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                        CONTROL PLANE                             │
│                                                                  │
│  ┌──────────────┐  ┌────────────────┐  ┌────────────────────┐   │
│  │ API Server   │  │  Scheduler     │  │Controller Manager|  |
│  |(kube-apiserver)| |(kube-scheduler)| |(kube-controller-|  |
│  |              |  |                |  |    manager)       |  |
│  └──────┬───────┘  └────────┬───────┘  └──────────┬─────────┘  |
│         |                   |                     |            |
│  ┌──────▼───────────────────▼─────────────────────▼──────────┐ |
│  |                    etcd (cluster database)                |  |
│  └──────────────────────────────────────────────────────────┘  |
└──────────────────────────────────────────────────────────────────┘
                          |  (API calls)
          ┌───────────────┼───────────────┐
          ▼               ▼               ▼

┌──────────────┐  ┌──────────────┐  ┌──────────────┐
|   WORKER     |  |   WORKER     |  |   WORKER     |
|   NODE 1     |  |   NODE 2     |  |   NODE 3     |
|              |  |              |  |              |
| ┌──────────┐ |  | ┌──────────┐ |  | ┌──────────┐ |
| | kubelet | |  | | kubelet | |  | | kubelet | |
```

```
| ├──────────┤ |   | ├──────────┤ |   | ├──────────┤ |
| |kube-     | |   | |kube-     | |   | |kube-     | |
| |proxy     | |   | |proxy     | |   | |proxy     | |
| ├──────────┤ |   | ├──────────┤ |   | ├──────────┤ |
| |Container| |   | |Container| |   | |Container| |
| |Runtime   | |   | |Runtime   | |   | |Runtime   | |
| └──────────┘ |   | └──────────┘ |   | └──────────┘ |
|              |   |              |   |              |
|  [Pod][Pod] |   |  [Pod][Pod] |   |  [Pod]      |
└──────────────┘   └──────────────┘   └──────────────┘
```

Control Plane Components

1. kube-apiserver

- The **front door** of Kubernetes
- All communication goes through it (kubectl, internal components, external tools)
- Validates and processes REST API requests
- Reads/writes state to `etcd`
- Stateless — can be replicated for high availability

```bash
# Every kubectl command hits the API server:
kubectl get pods
# → kubectl → HTTPS → kube-apiserver → etcd → response
```

2. etcd

- Distributed key-value store — the **single source of truth**
- Stores ALL cluster state: Pods, Services, Deployments, Secrets, etc.
- Uses the Raft consensus algorithm for consistency
- Should be backed up regularly in production

```bash
```

```
# etcd backup example:
ETCDCTL_API=3 etcdctl snapshot save snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

3. kube-scheduler

- Watches for newly created Pods with no assigned Node
- Selects the best Node based on:
  - Resource availability (CPU, Memory)
  - Node selectors and affinity rules
  - Taints and tolerations
  - Pod topology spread constraints

```
Scheduling pipeline:
  Filter phase  → removes nodes that can't run the Pod
  Score phase   → ranks remaining nodes (most free resources, etc.)
  Bind phase    → assigns Pod to the winning Node
```

4. kube-controller-manager

- Runs multiple controllers as a single binary
- Each controller watches the API server and reconciles state

Controller	What it manages
Node Controller	Detects when nodes go down
ReplicaSet Controller	Ensures correct number of Pod replicas
Deployment Controller	Manages rolling updates
Endpoints Controller	Populates Service endpoints
Namespace Controller	Manages namespace lifecycle
Job Controller	Manages batch Jobs to completion

- Integrates with cloud providers (AWS, GCP, Azure)
- Manages cloud-specific resources: Load Balancers, Node pools, Volumes

---

Worker Node Components

1. kubelet

- Agent running on **every Node**
- Receives PodSpecs from the API server
- Ensures containers described in PodSpecs are running and healthy
- Reports Node and Pod status back to API server
- Does NOT manage containers not created by K8s

2. kube-proxy

- Network proxy running on every Node
- Maintains network rules (iptables/ipvs) for Service routing
- Enables Pods to communicate across Nodes
- Routes traffic to the correct Pod when a Service is accessed

3. Container Runtime

- The actual software running containers
- K8s uses the CRI (Container Runtime Interface)
- Options: `containerd` (most common), `CRI-O`, `Docker` (deprecated in 1.24+)

How a Pod gets created — End to End

```
1. kubectl apply -f pod.yaml
         ↓
2. API Server validates YAML → stores in etcd
         ↓
3. Scheduler detects unscheduled Pod
         ↓
4. Scheduler selects best Node → updates etcd via API Server
```

```
        ↓
5. kubelet on selected Node detects new Pod assignment
        ↓
6. kubelet tells Container Runtime to pull image + start container
        ↓
7. kubelet reports status back to API Server → etcd updated
        ↓
8. Pod is Running ✓
```

---

3. Core Objects — Pods

What is a Pod?

- The **smallest deployable unit** in Kubernetes
- A wrapper around one or more containers
- Containers in a Pod share:
  - Network namespace (same IP address, same localhost)
  - Storage volumes
  - IPC namespace

Pod vs Container

```
Container: isolated process running an image (Docker concept)

Pod:
  ┌──────────────────────────────────────┐
  │  Shared Network (IP: 10.0.0.5)       │
  │  ┌──────────────┐ ┌──────────────┐   │
  │  │  Main container│ │  Sidecar    │   │
  │  │  (app:3000)    │ │  (nginx:80)│   │
  │  └──────────────┘ └──────────────┘   │
  │  Shared Volumes                      │
  └──────────────────────────────────────┘
```

Pod Design Patterns

Pattern	Description	Example
Single container	One app per Pod	web server

Pattern	Description	Example
Sidecar	Helper container alongside main	log shipper, proxy
Ambassador	Proxy to external services	DB connection pool
Adapter	Transforms output of main container	metrics exporter
Init containers	Run before main containers start	DB migration, wait for dependency

Simple Pod YAML

```yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  namespace: default
  labels:
    app: my-app
    environment: production
    version: "1.0"
  annotations:
    description: "Main application pod"
    team: "backend"
spec:
  containers:
    - name: my-app                        # container name
      image: nginx:1.25                   # image:tag
      ports:
        - containerPort: 80               # documents the port (informational)
      env:
        - name: APP_ENV
          value: "production"
        - name: DB_HOST
          value: "postgres-service"
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
  restartPolicy: Always                   # Always | OnFailure | Never
```

Pod with Init Container

```yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  # Init containers run SEQUENTIALLY before main containers start
  initContainers:
    - name: wait-for-db
      image: busybox:1.35
      command: ['sh', '-c',
        'until nslookup postgres-service; do echo waiting; sleep 2; done']

    - name: run-migrations
      image: my-app:latest
      command: ['python', 'manage.py', 'migrate']

  containers:
    - name: my-app
      image: my-app:latest
      ports:
        - containerPort: 8000
```

Pod with Sidecar Pattern

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  containers:
    # Main application container
    - name: app
      image: my-web-app:latest
      ports:
        - containerPort: 8080
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/app

    # Sidecar: ships logs to a log aggregator
    - name: log-shipper
      image: fluent/fluentd:v1.16
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/app
          readOnly: true

  volumes:
    - name: shared-logs
      emptyDir: {}         # shared ephemeral storage
```

Pod Lifecycle

```
Pending   → Scheduled, image being pulled, init containers running
Running   → At least one container is running
Succeeded → All containers exited with code 0 (Jobs)
Failed    → At least one container exited with non-zero code
Unknown   → Node communication lost
```

Pod Conditions

Condition	Meaning
`PodScheduled`	Pod assigned to a Node
`ContainersReady`	All containers are ready

Condition	Meaning
`Initialized`	All init containers completed
`Ready`	Pod can serve requests

Pod Commands

```bash
# Create a Pod
kubectl apply -f pod.yaml
kubectl run nginx --image=nginx          # imperative (quick testing)

# Inspect
kubectl get pods
kubectl get pods -o wide                 # shows Node, IP
kubectl describe pod my-app-pod
kubectl get pod my-app-pod -o yaml       # full YAML with status

# Execute commands
kubectl exec -it my-app-pod -- bash
kubectl exec -it my-app-pod -c sidecar -- sh   # specific container

# Logs
kubectl logs my-app-pod
kubectl logs my-app-pod -c log-shipper   # specific container
kubectl logs my-app-pod --previous       # crashed container logs
kubectl logs -f my-app-pod               # follow / stream

# Port forward (local testing)
kubectl port-forward pod/my-app-pod 8080:80

# Delete
kubectl delete pod my-app-pod
kubectl delete pod my-app-pod --grace-period=0  # force delete
```

---

4. ReplicaSet

What is a ReplicaSet?

- Ensures a **specified number of Pod replicas** are always running

- Replaces Pods that fail, get deleted, or are evicted
- Uses a **label selector** to identify which Pods it manages
- Rarely used directly — Deployments manage ReplicaSets for you

ReplicaSet YAML

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-app-rs
spec:
  replicas: 3                    # desired number of Pods
  selector:
    matchLabels:
      app: my-app                # selects Pods with this label
  template:                      # Pod template — how to create new Pods
    metadata:
      labels:
        app: my-app              # MUST match selector above
    spec:
      containers:
        - name: my-app
          image: nginx:1.25
          ports:
            - containerPort: 80
```

How ReplicaSet Works

```
Desired replicas: 3
Actual running:   1 (2 crashed)

ReplicaSet controller:
  → detects actual (1) ≠ desired (3)
  → creates 2 new Pods
  → actual now = 3 ✓

If you manually delete a Pod:
  → ReplicaSet immediately creates a replacement
```

ReplicaSet Commands

```bash
kubectl get replicasets
kubectl describe rs my-app-rs
kubectl scale rs my-app-rs --replicas=5
kubectl delete rs my-app-rs              # also deletes its Pods
kubectl delete rs my-app-rs --cascade=false  # keeps Pods, deletes RS
```

---

5. Deployments

What is a Deployment?

- The **most commonly used workload** in Kubernetes
- Manages ReplicaSets — wraps them with update/rollback capability
- Declaratively manages: desired replica count, Pod template, update strategy

Deployment → ReplicaSet → Pod hierarchy

```
Deployment (my-app-deployment)
    │
    ├── ReplicaSet v1 (my-app-deployment-7d9f4b8c6)  ← old (0 pods)
    │
    └── ReplicaSet v2 (my-app-deployment-6b8f7d9c1)  ← current (3 pods)
              │
              ├── Pod: my-app-deployment-6b8f7d9c1-abc12
              ├── Pod: my-app-deployment-6b8f7d9c1-def34
              └── Pod: my-app-deployment-6b8f7d9c1-ghi56
```

Full Deployment YAML

```yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  namespace: production
  labels:
    app: my-app
spec:
  replicas: 3

  selector:
    matchLabels:
      app: my-app

  # Rolling update strategy (default)
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1    # max pods that can be down during update
      maxSurge: 1          # max extra pods during update (above desired)

  template:
    metadata:
      labels:
        app: my-app
        version: "2.0"
    spec:
      containers:
        - name: my-app
          image: my-app:2.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "128Mi"
              cpu: "250m"
            limits:
              memory: "256Mi"
              cpu: "500m"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
```

```yaml
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
```

Update Strategies

RollingUpdate (default)

```
Before update: [v1][v1][v1]    replicas=3

With maxUnavailable=1, maxSurge=1:

Step 1:  [v1][v1][v1][v2]      surge: +1 new Pod
Step 2:  [v1][v1][v2]          remove 1 old
Step 3:  [v1][v1][v2][v2]      surge: +1 new
Step 4:  [v1][v2][v2]          remove 1 old
Step 5:  [v1][v2][v2][v2]      surge: +1 new
Step 6:  [v2][v2][v2]          remove last old ✓

Zero downtime — traffic always has live pods
```

Recreate

yaml

```yaml
strategy:
  type: Recreate       # Kill ALL old pods, then start new ones
                       # Causes downtime — use only when needed
```

Deployment Commands

bash

```
# Create / apply
kubectl apply -f deployment.yaml

# Scale
kubectl scale deployment my-app-deployment --replicas=5

# Update image (triggers rolling update)
kubectl set image deployment/my-app-deployment my-app=my-app:3.0

# Check rollout status
kubectl rollout status deployment/my-app-deployment

# View rollout history
kubectl rollout history deployment/my-app-deployment
kubectl rollout history deployment/my-app-deployment --revision=2

# Rollback to previous version
kubectl rollout undo deployment/my-app-deployment

# Rollback to specific revision
kubectl rollout undo deployment/my-app-deployment --to-revision=2

# Pause a rollout (e.g. to batch changes)
kubectl rollout pause deployment/my-app-deployment
kubectl set image deployment/my-app-deployment my-app=my-app:4.0
kubectl set env deployment/my-app-deployment DB_VERSION=14
kubectl rollout resume deployment/my-app-deployment

# Restart all pods in deployment
kubectl rollout restart deployment/my-app-deployment
```

---

6. Services

What is a Service?

- Provides a **stable network endpoint** to access Pods
- Pods have ephemeral IPs that change on restart — Services don't
- Load balances traffic across matching Pods using label selectors
- Assigned a stable DNS name in the cluster

```
Without Service:
  Client → Pod IP (10.0.0.5) → Pod deleted → IP gone → ❌ broken

With Service:
  Client → Service IP (10.96.5.100) → Service finds live Pods → ✓
  (Service IP never changes, Pods come and go freely)
```

Service Types

1. ClusterIP (default)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  type: ClusterIP        # accessible ONLY within the cluster
  selector:
    app: my-app          # forwards traffic to pods with this label
  ports:
    - port: 80           # Service port (what clients connect to)
      targetPort: 8080   # Pod port (what the container listens on)
      protocol: TCP
```

```
Client Pod → my-app-service:80 → [Pod1:8080] [Pod2:8080] [Pod3:8080]
```

2. NodePort

```yaml
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-nodeport
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
    - port: 80           # ClusterIP port
      targetPort: 8080   # Pod port
      nodePort: 30080    # External port on every Node (30000-32767)
```

```
External user → http://NodeIP:30080 → Service:80 → Pod:8080
(Accessible from outside the cluster via any Node's IP)
```

3. LoadBalancer

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-lb
spec:
  type: LoadBalancer     # provisions cloud load balancer (AWS ALB, GCP LB)
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
```

```
External → Cloud LB (public IP) → NodePort → Pod
(Only works on cloud providers; use MetalLB for on-prem)
```

4. ExternalName

```yaml
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: prod-db.example.com  # maps to external DNS name
```

```
Pod → external-db (K8s DNS) → prod-db.example.com (outside cluster)
(CNAME redirect — no selector, no load balancing)
```

Service Discovery and DNS

```bash
# Every Service gets a DNS record in the cluster:
# Format: <service-name>.<namespace>.svc.cluster.local

# From within the same namespace:
curl http://my-app-service

# From a different namespace:
curl http://my-app-service.production.svc.cluster.local

# DNS resolution example:
nslookup my-app-service.default.svc.cluster.local
# → 10.96.5.100 (ClusterIP)
```

Headless Service (for StatefulSets)

```yaml
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-statefulset-svc
spec:
  clusterIP: None          # headless — no virtual IP, returns Pod IPs directly
  selector:
    app: my-statefulset
  ports:
    - port: 5432
```

```
DNS returns individual Pod IPs instead of a single ClusterIP.
Used for StatefulSets where you need to address specific pods:
  pod-0.my-statefulset-svc.default.svc.cluster.local → 10.0.1.1
  pod-1.my-statefulset-svc.default.svc.cluster.local → 10.0.1.2
```

Endpoints

```bash
# Services have an Endpoints object showing which Pods are selected
kubectl get endpoints my-app-service

# Output:
# NAME             ENDPOINTS                         AGE
# my-app-service   10.0.0.5:8080,10.0.0.6:8080     5m
```

---

7. Namespaces

What is a Namespace?

- A **virtual cluster** within a Kubernetes cluster
- Provides resource isolation between teams/environments
- Resource names must be unique within a namespace, not across namespaces
- Resources like Nodes and PersistentVolumes are cluster-scoped (not namespaced)

Default Namespaces

Namespace	Purpose
`default`	Default for user workloads
`kube-system`	K8s internal components (API server, DNS, etc.)
`kube-public`	Readable by all users, used for public cluster info
`kube-node-lease`	Node heartbeat objects

Namespace YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: staging
  labels:
    environment: staging
    team: backend
```

Resource Quotas per Namespace

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: staging-quota
  namespace: staging
spec:
  hard:
    pods: "20"                  # max 20 pods
    requests.cpu: "4"           # max 4 CPU cores requested
    requests.memory: 8Gi        # max 8 GB memory requested
    limits.cpu: "8"             # max 8 CPU cores limit
    limits.memory: 16Gi         # max 16 GB memory limit
    services: "10"              # max 10 services
    persistentvolumeclaims: "5" # max 5 PVCs
```

LimitRange (default resource limits)

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: staging
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"             # default limit if not specified
        memory: "256Mi"
      defaultRequest:
        cpu: "200m"             # default request if not specified
        memory: "128Mi"
      max:
        cpu: "2"                # containers can't exceed this
        memory: "1Gi"
      min:
        cpu: "50m"              # containers must request at least this
        memory: "32Mi"
```

Namespace Commands

```bash
```

```
# Create
kubectl create namespace production
kubectl apply -f namespace.yaml

# List all namespaces
kubectl get namespaces

# Work in a specific namespace
kubectl get pods -n production
kubectl apply -f deployment.yaml -n production

# Set default namespace for current context
kubectl config set-context --current --namespace=production

# Get resources across ALL namespaces
kubectl get pods --all-namespaces
kubectl get pods -A       # shorthand
```

---

8. ConfigMaps & Secrets

ConfigMap

Stores **non-sensitive configuration** as key-value pairs, injected into Pods as environment variables or mounted as files.

Create ConfigMap

```yaml
```

```yaml
# Method 1: Literal values
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  APP_PORT: "8080"
  LOG_LEVEL: "info"
  MAX_CONNECTIONS: "100"

  # Multi-line value (config file)
  app.properties: |
    server.port=8080
    spring.datasource.url=jdbc:postgresql://db:5432/mydb
    spring.jpa.hibernate.ddl-auto=validate
```

bash

```bash
# Method 2: From command line
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info

# Method 3: From a file
kubectl create configmap app-config --from-file=app.properties

# Method 4: From a directory
kubectl create configmap app-config --from-file=./config/
```

Use ConfigMap in Pod — Environment Variables

yaml

```yaml
spec:
  containers:
    - name: my-app
      image: my-app:latest
      env:
        # Single value from ConfigMap
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV

      # All values from ConfigMap as env vars
      envFrom:
        - configMapRef:
            name: app-config
```

Use ConfigMap in Pod — Volume Mount (as files)

```yaml
spec:
  containers:
    - name: my-app
      image: my-app:latest
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config    # files appear here in the container

  volumes:
    - name: config-volume
      configMap:
        name: app-config            # each key becomes a file
```

Secrets

Stores **sensitive data** (passwords, tokens, keys) base64-encoded (not truly encrypted by default — use etcd encryption at rest for real security).

Secret Types

Type	Use case
`Opaque`	Generic key-value pairs (default)
`kubernetes.io/dockerconfigjson`	Docker registry credentials
`kubernetes.io/tls`	TLS certificate and key
`kubernetes.io/service-account-token`	Service account token

Create Secret

yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  # Values must be base64-encoded
  DB_PASSWORD: cGFzc3dvcmQxMjM=   # echo -n "password123" | base64
  DB_USER: YWRtaW4=                # echo -n "admin" | base64
stringData:
  # stringData is auto-encoded — more convenient
  DB_HOST: "postgres-service"
  DB_NAME: "mydb"
```

bash

```
# Create from command line
kubectl create secret generic db-secret \
  --from-literal=DB_PASSWORD=password123 \
  --from-literal=DB_USER=admin

# Create TLS secret
kubectl create secret tls my-tls \
  --cert=tls.crt \
  --key=tls.key

# Create Docker registry secret
kubectl create secret docker-registry regcred \
  --docker-server=docker.io \
  --docker-username=myuser \
  --docker-password=mypassword
```

Use Secret in Pod

```yaml
spec:
  containers:
    - name: my-app
      image: my-app:latest
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: DB_PASSWORD
      envFrom:
        - secretRef:
            name: db-secret        # all keys as env vars

  # Pull private image using secret
  imagePullSecrets:
    - name: regcred
```

ConfigMap vs Secret

	ConfigMap	Secret
Data type	Plain text	Base64 encoded

	ConfigMap	Secret
For	Config values, app settings	Passwords, tokens, keys
In memory	Stored in etcd plaintext	Stored in etcd (encrypt at rest!)
Volume	Yes	Yes (tmpfs — not written to disk)
Env vars	Yes	Yes
Size limit	1 MB	1 MB

9. Volumes & Persistent Storage

Why Volumes?

Container filesystems are **ephemeral** — data is lost when the container restarts. Volumes provide persistent or shared storage.

Volume Types

emptyDir — Ephemeral Shared Storage

```yaml
spec:
  volumes:
    - name: cache-vol
      emptyDir:
        medium: Memory   # RAM-backed (optional) — empty = disk
        sizeLimit: 500Mi

  containers:
    - name: app
      volumeMounts:
        - name: cache-vol
          mountPath: /cache
```

```
- Created when Pod starts, deleted when Pod is removed
- Shared between containers in the same Pod
- Survives container restarts (but NOT Pod deletion)
- Use case: cache, scratch space, shared data between sidecars
```

```yaml
spec:
  volumes:
    - name: host-vol
      hostPath:
        path: /var/log/app
        type: DirectoryOrCreate

  containers:
    - name: app
      volumeMounts:
        - name: host-vol
          mountPath: /logs
```

```
- Mounts a directory from the Node's filesystem
- Persists as long as the Node exists
- Not portable — Pod tied to one Node
- Security risk — avoid unless necessary
```

configMap and secret as volumes

```yaml
# Already covered in ConfigMaps & Secrets section
volumes:
  - name: config-vol
    configMap:
      name: app-config
  - name: secret-vol
    secret:
      secretName: db-secret
      defaultMode: 0400  # file permissions
```

Persistent Volumes (PV) and Persistent Volume Claims (PVC)

The **two-step storage pattern** that decouples storage provisioning from usage:

```
Admin creates PV (or StorageClass auto-provisions)
  ↓
Developer creates PVC (requests storage)
  ↓
K8s binds PVC to a matching PV
  ↓
Pod mounts the PVC
```

PersistentVolume (PV) — admin-created storage

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce        # RWO: one node read/write
    # - ReadOnlyMany       # ROX: many nodes read-only
    # - ReadWriteMany      # RWX: many nodes read/write (NFS, EFS)
  persistentVolumeReclaimPolicy: Retain  # Retain | Recycle | Delete
  storageClassName: standard
  hostPath:                              # storage backend (example)
    path: /mnt/data
```

PersistentVolumeClaim (PVC) — developer's request

```yaml
```

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi          # request 5 GB (will bind to 10Gi PV above)
  storageClassName: standard
```

Mount PVC in a Pod

```yaml
spec:
  containers:
    - name: database
      image: postgres:15
      volumeMounts:
        - name: db-storage
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: db-storage
      persistentVolumeClaim:
        claimName: my-pvc   # reference to the PVC
```

StorageClass (Dynamic Provisioning)

```yaml
```

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs  # cloud-specific provisioner
parameters:
  type: gp3
  fsType: ext4
  encrypted: "true"
reclaimPolicy: Delete               # Delete | Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

```bash
# Use in PVC:
spec:
  storageClassName: fast-ssd   # K8s auto-provisions an EBS volume
```

Access Modes Summary

Mode	Short	Description
ReadWriteOnce	RWO	One node can read and write
ReadOnlyMany	ROX	Multiple nodes can read
ReadWriteMany	RWX	Multiple nodes can read and write
ReadWriteOncePod	RWOP	Only one Pod can read and write

---

10. StatefulSets

What is a StatefulSet?

Used for **stateful applications** that need:

- Stable, predictable Pod names (pod-0, pod-1, pod-2)
- Stable network identities (persistent DNS names)
- Ordered deployment and scaling

- Persistent storage per Pod (each Pod gets its own PVC)

StatefulSet vs Deployment

Feature	Deployment	StatefulSet
Pod names	Random hash	Ordered (pod-0, pod-1)
Scaling	Simultaneous	Ordered (one at a time)
Storage	Shared PVC	Individual PVC per pod
Network identity	Ephemeral	Stable DNS per pod
Use case	Stateless apps	Databases, Kafka, Zookeeper

StatefulSet YAML

```yaml
```

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless     # headless service name (required)
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: password
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data

  # Each Pod gets its own PVC — created automatically
  volumeClaimTemplates:
    - metadata:
        name: postgres-data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 20Gi
---
# Headless service for stable DNS
apiVersion: v1
```

```yaml
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
```

StatefulSet DNS Pattern

```
Pod name:     postgres-0, postgres-1, postgres-2
DNS pattern:  <pod-name>.<service-name>.<namespace>.svc.cluster.local

postgres-0.postgres-headless.default.svc.cluster.local
postgres-1.postgres-headless.default.svc.cluster.local
postgres-2.postgres-headless.default.svc.cluster.local

PVCs created: postgres-data-postgres-0
              postgres-data-postgres-1
              postgres-data-postgres-2
```

---

11. DaemonSets

What is a DaemonSet?

Ensures that **one copy of a Pod runs on every Node** (or a subset of nodes). When a new Node joins the cluster, a Pod is automatically added. When a Node is removed, the Pod is garbage-collected.

Use Cases

- Log collection agents (Fluentd, Filebeat)
- Monitoring agents (Prometheus node-exporter, Datadog agent)
- Node-level networking (Calico, Cilium)
- Storage daemons (Ceph, GlusterFS)

DaemonSet YAML

yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      # Run on every node (including control plane)
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule

      containers:
        - name: node-exporter
          image: prom/node-exporter:latest
          ports:
            - containerPort: 9100
          volumeMounts:
            - name: proc
              mountPath: /host/proc
              readOnly: true
            - name: sys
              mountPath: /host/sys
              readOnly: true

      volumes:
        - name: proc
          hostPath:
            path: /proc
        - name: sys
          hostPath:
            path: /sys

      # Allow scheduling on every node
```

```yaml
      hostPID: true
      hostNetwork: true
```

---

12. Jobs & CronJobs

Job

Runs a **batch task to completion** — ensures a specified number of Pods successfully terminate.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  completions: 1          # number of successful completions required
  parallelism: 1          # max pods running simultaneously
  backoffLimit: 3         # retry up to 3 times on failure
  activeDeadlineSeconds: 300  # kill job if not done in 5 min
  template:
    spec:
      restartPolicy: OnFailure   # Never | OnFailure (not Always)
      containers:
        - name: migrate
          image: my-app:latest
          command: ["python", "manage.py", "migrate"]
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: DB_PASSWORD
```

Parallel Job Patterns

```yaml
```

```yaml
# Pattern 1: Fixed completion count (N workers, each does 1 task)
spec:
  completions: 10
  parallelism: 3          # 3 pods at a time, runs until 10 succeed

# Pattern 2: Work queue (pods consume from queue until empty)
spec:
  parallelism: 5          # 5 workers, no fixed completion count
  completions: 1          # job done when any 1 pod succeeds
```

CronJob

Runs Jobs on a **schedule** (like Linux cron).

```yaml
```

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-report
spec:
  schedule: "0 6 * * *"         # cron expression: 6 AM every day
  # ┌──── minute (0-59)
  # │ ┌── hour (0-23)
  # │ │ ┌ day of month (1-31)
  # │ │ │ ┌ month (1-12)
  # │ │ │ │ ┌ day of week (0-7, 0=Sun)
  # 0 6 * * *

  concurrencyPolicy: Forbid     # Allow | Forbid | Replace
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  startingDeadlineSeconds: 60   # skip if missed by more than 60s

  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: my-reports:latest
              command: ["python", "generate_daily_report.py"]
```

CronJob Schedule Examples

```
"* * * * *"        Every minute
"*/5 * * * *"      Every 5 minutes
"0 * * * *"        Every hour at :00
"0 0 * * *"        Daily at midnight
"0 6 * * 1-5"      Weekdays at 6 AM
"0 0 1 * *"        First day of every month
"0 0 * * 0"        Every Sunday at midnight
```

---

13. Ingress & Ingress Controllers

What is Ingress?

- Manages **external HTTP/HTTPS access** to Services within the cluster
- Provides: host-based routing, path-based routing, TLS termination, redirects
- An Ingress resource is just rules — it needs an **Ingress Controller** to act on them

Ingress Controller

A Pod (reverse proxy) that reads Ingress rules and configures itself accordingly. Popular options: **nginx-ingress**, Traefik, HAProxy, AWS ALB Ingress Controller

Install nginx-ingress (Helm)

```bash
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace
```

Path-Based Routing

```yaml
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

Host-Based Routing (Virtual hosting)

yaml

```yaml
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-service
                port:
                  number: 80

    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 80
```

TLS/HTTPS Ingress

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: secure-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod   # auto-issue certs
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - myapp.example.com
      secretName: myapp-tls-secret   # K8s secret with cert+key

  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-service
                port:
                  number: 80
```

---

14. Resource Management

Requests vs Limits

```
Requests: what the container is GUARANTEED (used for scheduling)
Limits:   what the container is ALLOWED to use at most

CPU is compressible  → throttled when exceeding limit (no kill)
Memory is not        → container killed (OOMKilled) if exceeds limit
```

```yaml
```

```yaml
resources:
  requests:
    memory: "128Mi"    # scheduler uses this to find a node
    cpu: "250m"        # 250 millicores = 0.25 CPU cores
  limits:
    memory: "256Mi"    # max memory (OOMKilled if exceeded)
    cpu: "500m"        # max CPU (throttled, not killed)
```

CPU Units

```
1 CPU = 1 vCPU = 1 core = 1000m (millicores)

250m  = 0.25 CPU
500m  = 0.5 CPU
1     = 1 CPU
2     = 2 CPU
```

Quality of Service (QoS) Classes

QoS Class	Condition	Eviction order
Guaranteed	requests == limits for all containers	Last to be evicted
Burstable	Some containers have requests < limits	Middle
BestEffort	No requests or limits set	First to be evicted

```yaml
# Guaranteed QoS
resources:
  requests:
    cpu: "500m"
    memory: "256Mi"
  limits:
    cpu: "500m"        # same as request
    memory: "256Mi"    # same as request

# BestEffort QoS
# (no resources block at all — bad practice for production!)
```

15. Health Checks — Probes

Three Types of Probes

Probe	Purpose	Failure action
livenessProbe	Is the container alive?	Container restarted
readinessProbe	Is the container ready for traffic?	Removed from Service endpoints
startupProbe	Has the container finished starting?	Liveness/readiness disabled until passes

Probe Mechanisms

```yaml
```

```yaml
# HTTP GET probe — most common for web services
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
    httpHeaders:
      - name: Authorization
        value: Bearer token123
  initialDelaySeconds: 15    # wait 15s before first check
  periodSeconds: 10           # check every 10s
  timeoutSeconds: 5           # fail if no response in 5s
  failureThreshold: 3         # restart after 3 consecutive failures
  successThreshold: 1         # 1 success = healthy

# TCP Socket probe — for TCP servers
readinessProbe:
  tcpSocket:
    port: 5432
  initialDelaySeconds: 5
  periodSeconds: 10

# Exec probe — run a command
livenessProbe:
  exec:
    command:
      - sh
      - -c
      - "pg_isready -U postgres"
  initialDelaySeconds: 10
  periodSeconds: 15

# gRPC probe (K8s 1.24+)
livenessProbe:
  grpc:
    port: 50051
    service: my-service
```

Full Probe Example

yaml

```yaml
spec:
  containers:
    - name: my-app
      image: my-app:latest

      # startupProbe: give slow-starting apps extra time
      startupProbe:
        httpGet:
          path: /health/startup
          port: 8080
        failureThreshold: 30    # 30 × 10s = 5 min to start
        periodSeconds: 10

      # readinessProbe: remove from LB if not ready
      readinessProbe:
        httpGet:
          path: /health/ready
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 3

      # livenessProbe: restart if hung/deadlocked
      livenessProbe:
        httpGet:
          path: /health/live
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
        failureThreshold: 3
```

---

16. Horizontal & Vertical Pod Autoscaling

Horizontal Pod Autoscaler (HPA)

Scales the **number of Pod replicas** based on metrics (CPU, memory, custom).

```yaml
```

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app-deployment
  minReplicas: 2
  maxReplicas: 20
  metrics:
    # CPU-based scaling
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70    # scale up when avg CPU > 70%

    # Memory-based scaling
    - type: Resource
      resource:
        name: memory
        target:
          type: AverageValue
          averageValue: 200Mi

    # Custom metric (e.g. requests per second)
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "100"
```

```bash
```

```
# Quick HPA creation
kubectl autoscale deployment my-app --min=2 --max=10 --cpu-percent=70

# Check HPA status
kubectl get hpa
kubectl describe hpa my-app-hpa
```

Vertical Pod Autoscaler (VPA)

Adjusts **CPU and Memory requests/limits** automatically.

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app-deployment
  updatePolicy:
    updateMode: "Auto"    # Auto | Off | Initial | Recreate
  resourcePolicy:
    containerPolicies:
      - containerName: my-app
        minAllowed:
          cpu: 100m
          memory: 128Mi
        maxAllowed:
          cpu: 2
          memory: 2Gi
```

HPA vs VPA

	HPA	VPA
Scales	Pod count	Pod resource requests
Metric types	CPU, memory, custom	CPU, memory
Use with	Stateless apps	Stateful apps

	HPA	VPA
Can run together?	Yes (carefully — don't scale on CPU if VPA changes CPU)	—

17. RBAC — Role-Based Access Control

Core Concepts

```
Subject (WHO)    → can perform →  Verbs (WHAT)    → on →  Resources (WHICH)
User/Group/SA                      get/list/create          pods/services/secrets
```

RBAC Objects

Object	Scope	Description
`Role`	Namespaced	Defines permissions within a namespace
`ClusterRole`	Cluster-wide	Defines permissions across all namespaces
`RoleBinding`	Namespaced	Binds Role/ClusterRole to subjects in a namespace
`ClusterRoleBinding`	Cluster-wide	Binds ClusterRole to subjects cluster-wide

Role Example

yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
  - apiGroups: [""]              # "" = core API group
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]

  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
```

RoleBinding Example

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: production
subjects:
  # Bind to a User
  - kind: User
    name: alice
    apiGroup: rbac.authorization.k8s.io

  # Bind to a Group
  - kind: Group
    name: dev-team
    apiGroup: rbac.authorization.k8s.io

  # Bind to a ServiceAccount
  - kind: ServiceAccount
    name: my-app-sa
    namespace: production

roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ClusterRole Example

```yaml
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]

  - apiGroups: [""]
    resources: ["namespaces"]
    verbs: ["get", "list"]
```

ServiceAccount

yaml

```yaml
# Create a ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: production

# Use it in a Pod
spec:
  serviceAccountName: my-app-sa
  containers:
    - name: my-app
      image: my-app:latest
```

RBAC Verbs Reference

```
get        → read a single resource
list       → read all resources of a type
watch      → stream changes to resources
create     → create a resource
update     → replace a resource
patch      → partially modify a resource
delete     → delete a resource
deletecollection → delete all resources of a type
exec       → exec into a pod (pods/exec)
```

Check Permissions

```bash
# Can current user list pods in default namespace?
kubectl auth can-i list pods

# Can alice create deployments in production?
kubectl auth can-i create deployments --namespace=production --as=alice

# List all permissions for current user
kubectl auth can-i --list

# List all permissions for a ServiceAccount
kubectl auth can-i --list --as=system:serviceaccount:production:my-app-sa
```

---

18. Network Policies

What is a NetworkPolicy?

Controls **network traffic** between Pods (firewall rules at the Pod level). By default, all Pods can communicate with each other — NetworkPolicy restricts this.

> Requires a CNI plugin that supports NetworkPolicy (Calico, Cilium, Weave).

Deny All Ingress (Default deny)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: production
spec:
  podSelector: {}       # applies to ALL pods in namespace
  policyTypes:
    - Ingress
  # No ingress rules = deny all ingress traffic
```

Allow Specific Traffic

```yaml
```

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend        # this policy applies to backend pods

  policyTypes:
    - Ingress
    - Egress

  ingress:
    # Allow traffic FROM frontend pods
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080

    # Allow traffic FROM monitoring namespace
    - from:
        - namespaceSelector:
            matchLabels:
              purpose: monitoring
      ports:
        - protocol: TCP
          port: 9090

  egress:
    # Allow traffic TO database pods
    - to:
        - podSelector:
            matchLabels:
              app: postgres
      ports:
        - protocol: TCP
          port: 5432
```

```yaml
    # Allow DNS resolution
    - to: []
      ports:
        - protocol: UDP
          port: 53
```

---

19. Helm — Package Manager

What is Helm?

Helm is the **package manager for Kubernetes** — it packages K8s manifests into reusable "charts".

```
Chart:    a Helm package (like a Docker image for K8s apps)
Release:  an installed instance of a chart
Repository: a collection of charts (like Docker Hub)
Values:   configuration variables for a chart
```

Helm Commands

```bash
```

```bash
# Add a chart repository
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for charts
helm search repo postgres
helm search hub nginx

# Install a chart
helm install my-postgres bitnami/postgresql

# Install with custom values
helm install my-postgres bitnami/postgresql \
  --set auth.postgresPassword=mysecret \
  --set primary.persistence.size=20Gi

# Install from a values file
helm install my-postgres bitnami/postgresql -f custom-values.yaml

# List installed releases
helm list
helm list -n production

# Upgrade a release
helm upgrade my-postgres bitnami/postgresql --set image.tag=15.3.0

# Rollback a release
helm rollback my-postgres 1   # roll back to revision 1

# Uninstall a release
helm uninstall my-postgres

# Show chart info
helm show chart bitnami/postgresql
helm show values bitnami/postgresql    # all configurable values
```

Creating a Helm Chart

```bash
```

```
helm create my-app-chart
# Creates:
# my-app-chart/
#   Chart.yaml         ← chart metadata
#   values.yaml        ← default values
#   templates/         ← K8s manifest templates
#     deployment.yaml
#     service.yaml
#     ingress.yaml
#     _helpers.tpl      ← template helpers
```

Chart.yaml

yaml

```yaml
apiVersion: v2
name: my-app-chart
description: My Application Helm Chart
type: application
version: 1.2.0          # Chart version
appVersion: "2.0.0"     # App version being deployed
```

values.yaml

yaml

```yaml
replicaCount: 3
image:
  repository: my-app
  tag: "2.0.0"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources:
  requests:
    cpu: 250m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
ingress:
  enabled: true
  host: myapp.example.com
```

Template (templates/deployment.yaml)

```yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-app-chart.fullname" . }}
  labels:
    {{- include "my-app-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "my-app-chart.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "my-app-chart.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

---

20. Monitoring & Logging

Metrics Stack: Prometheus + Grafana

```
[Pods] → expose /metrics → [Prometheus scrapes] → [Grafana visualises]
```

```bash
# Install kube-prometheus-stack (Helm)
helm repo add prometheus-community https://prometheus-community.github.io/helm-chart
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

Logging Stack: EFK (Elasticsearch + Fluentd + Kibana)

```
[Pods] → stdout/stderr → [Fluentd DaemonSet collects] → [Elasticsearch stores] →
[Kibana displays]
```

kubectl Logging Commands

```bash
# Basic logs
kubectl logs pod-name
kubectl logs pod-name -c container-name    # multi-container pod

# Stream logs
kubectl logs -f pod-name

# Previous container logs (after crash)
kubectl logs pod-name --previous

# Last N lines
kubectl logs pod-name --tail=100

# Since time
kubectl logs pod-name --since=1h
kubectl logs pod-name --since-time=2025-01-01T06:00:00Z

# All pods with label
kubectl logs -l app=my-app --all-containers

# Events (great for debugging)
kubectl get events --sort-by=.lastTimestamp
kubectl get events -n production --field-selector type=Warning
```

Key Metrics to Monitor

```
Pod level:
  cpu_usage, memory_usage, restart_count, ready

Node level:
  node_cpu_usage, node_memory_usage, node_disk_usage

Cluster level:
```

```
  pod_count, deployment_replicas, pvc_usage

Application level:
  http_request_rate, error_rate, latency_p99
```

---

21. Advanced Scheduling

Node Selectors

```yaml
# Simple: schedule on nodes with this label
spec:
  nodeSelector:
    disktype: ssd
    region: us-east-1
```

Node Affinity (more expressive)

```yaml
spec:
  affinity:
    nodeAffinity:
      # Hard requirement — Pod won't schedule without it
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/arch
                operator: In
                values: ["amd64", "arm64"]

      # Soft preference — try to schedule here, but not required
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd"]
```

Pod Affinity and Anti-Affinity

```yaml
spec:
  affinity:
    # Spread web-server pods away from each other (anti-affinity)
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: web-server
          topologyKey: kubernetes.io/hostname  # one pod per node

    # Schedule app near its cache (affinity)
    podAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: redis-cache
            topologyKey: kubernetes.io/hostname
```

Taints and Tolerations

```bash
# Taint a node (mark it as "special")
kubectl taint nodes gpu-node gpu=true:NoSchedule
# NoSchedule     → don't schedule pods without toleration
# PreferNoSchedule → prefer not to (soft)
# NoExecute      → evict existing pods without toleration

# Remove taint
kubectl taint nodes gpu-node gpu=true:NoSchedule-
```

```yaml
```

```yaml
# Toleration in Pod spec — "I can tolerate this taint"
spec:
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

  nodeSelector:
    gpu: "true"           # also select GPU nodes
```

Topology Spread Constraints

```yaml
spec:
  topologySpreadConstraints:
    - maxSkew: 1           # max difference in pod count between zones
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule    # hard: don't schedule
      labelSelector:
        matchLabels:
          app: my-app
```

---

22. Custom Resource Definitions (CRDs)

What are CRDs?

Extend the Kubernetes API with **custom resource types**. Operators use CRDs to manage complex stateful applications.

```yaml
```

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.mycompany.io
spec:
  group: mycompany.io
  names:
    kind: Database
    plural: databases
    singular: database
    shortNames: ["db"]
  scope: Namespaced
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                  enum: ["postgres", "mysql"]
                version:
                  type: string
                replicas:
                  type: integer
                  minimum: 1
```

yaml

```yaml
# Use the custom resource
apiVersion: mycompany.io/v1
kind: Database
metadata:
  name: my-postgres-db
spec:
  engine: postgres
  version: "15.3"
  replicas: 3
```

Kubernetes Operators

An **Operator** = CRD + Controller. It automates complex operational tasks using domain knowledge encoded in software.

Examples:

- `prometheus-operator` — manages Prometheus instances
- `postgres-operator` — manages PostgreSQL clusters
- `cert-manager` — automatically provisions TLS certificates

---

23. kubectl — Complete Command Reference

Context and Config

```bash
# View config
kubectl config view
kubectl config get-contexts
kubectl config current-context

# Switch context
kubectl config use-context my-cluster

# Set namespace for current context
kubectl config set-context --current --namespace=production

# Merge kubeconfigs
KUBECONFIG=~/.kube/config:~/.kube/config2 kubectl config view --merge --flatten
```

Get Resources

```bash
kubectl get pods
kubectl get pods -n kube-system
kubectl get pods -A                      # all namespaces
kubectl get pods -o wide                 # extra columns
kubectl get pods -o yaml                 # full YAML
kubectl get pods -o json                 # full JSON
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -w                      # watch for changes
kubectl get pods --field-selector=status.phase=Running
kubectl get pods -l app=my-app           # label selector
kubectl get all                          # everything in namespace
kubectl get all -A                       # everything cluster-wide
```

Create and Delete

```bash
kubectl apply -f manifest.yaml
kubectl apply -f ./manifests/            # apply all files in directory
kubectl apply -f https://example.com/manifest.yaml
kubectl delete -f manifest.yaml
kubectl delete pod my-pod
kubectl delete pod my-pod --grace-period=0 --force
kubectl delete pods -l app=my-app        # by label
```

Describe and Debug

```bash
```

```bash
kubectl describe pod my-pod
kubectl describe node my-node
kubectl describe service my-service

kubectl exec -it my-pod -- bash
kubectl exec -it my-pod -c sidecar -- sh

kubectl cp my-pod:/var/log/app.log ./app.log    # copy from pod
kubectl cp ./config.yaml my-pod:/etc/config.yaml  # copy to pod

kubectl port-forward pod/my-pod 8080:80
kubectl port-forward service/my-service 8080:80
kubectl port-forward deployment/my-app 8080:80

kubectl top pods
kubectl top nodes
kubectl top pod my-pod --containers
```

Editing Resources

```bash
kubectl edit deployment my-app          # open in $EDITOR
kubectl patch deployment my-app -p '{"spec":{"replicas":5}}'
kubectl patch deployment my-app --type=json \
  -p='[{"op":"replace","path":"/spec/replicas","value":5}]'
```

Labels and Annotations

```bash
# Add / update label
kubectl label pod my-pod env=prod
kubectl label pod my-pod env=staging --overwrite

# Remove label
kubectl label pod my-pod env-

# Add annotation
kubectl annotate pod my-pod \
  description="This is the main app pod"
```

Useful Aliases

```bash
alias k='kubectl'
alias kgp='kubectl get pods'
alias kgs='kubectl get services'
alias kgd='kubectl get deployments'
alias kaf='kubectl apply -f'
alias kdel='kubectl delete'
alias kex='kubectl exec -it'
alias klog='kubectl logs -f'
alias kctx='kubectl config use-context'
alias kns='kubectl config set-context --current --namespace'
```

---

24. Top 60 Interview Questions & Answers

Beginner Level

Q1. What is Kubernetes and why is it used?

> Kubernetes is an open-source container orchestration platform that automates deployment, scaling, and management of containerised applications. It is used because it provides self-healing (restarts failed containers), auto-scaling, rolling updates, service discovery, and declarative configuration management.

Q2. What is the difference between a Pod and a Container?

> A Container is a running process from a Docker image. A Pod is the smallest K8s unit — it wraps one or more containers that share a network namespace (IP address) and storage volumes. Containers in the same Pod can communicate via localhost.

Q3. What is a Node in Kubernetes?

> A Node is a physical or virtual machine that runs containerised workloads. Each Node runs a `kubelet` (agent), `kube-proxy` (network proxy), and a Container Runtime (containerd).

Q4. What is etcd?

> etcd is a distributed key-value store used as the Kubernetes backing store for all cluster data. It stores the desired state of every K8s object. It uses the Raft consensus algorithm to ensure consistency across replicas. It must be backed up regularly.

Q5. What does the kube-scheduler do?

> The kube-scheduler watches for newly created Pods with no assigned Node. It selects the best Node based on resource availability, node affinity/anti-affinity rules, taints and tolerations, and topology constraints.

Q6. What is a Deployment?

> A Deployment is the most common K8s workload controller. It manages a set of identical Pods via a ReplicaSet, enabling rolling updates, rollbacks, scaling, and desired-state management of stateless applications.

Q7. What is the difference between RollingUpdate and Recreate strategies?

> `RollingUpdate` gradually replaces old Pods with new ones — zero downtime. `Recreate` terminates all old Pods before starting new ones — causes downtime but ensures no two versions run simultaneously.

Q8. What are Labels and Selectors?

> Labels are key-value pairs attached to K8s objects for identification. Selectors filter objects by their labels. Services use selectors to route traffic to Pods; ReplicaSets use them to identify which Pods they manage.

Q9. What is a Namespace?

> A Namespace is a virtual cluster within a Kubernetes cluster providing resource isolation between teams or environments. Resource names must be unique within a namespace but can duplicate across namespaces.

Q10. What is the difference between kubectl apply and kubectl create?

> `kubectl create` creates a resource only if it doesn't exist — fails if it already does. `kubectl apply` is declarative — creates the resource if new, or updates it if it already exists. `apply` is preferred for infrastructure-as-code workflows.

Intermediate Level

Q11. What is the difference between a StatefulSet and a Deployment?

> Deployments are for stateless apps — Pods have random names, shared storage, and are interchangeable. StatefulSets are for stateful apps — Pods have stable ordered names (pod-0, pod-1), individual PVCs, stable DNS names, and ordered scaling.

Q12. What is a DaemonSet and when would you use it?

> A DaemonSet ensures one Pod runs on every Node (or a subset). Use cases: log collection agents (Fluentd), node monitoring (node-exporter), networking plugins (Calico, Cilium), and storage daemons.

Q13. What is the difference between livenessProbe and readinessProbe?

> `livenessProbe` detects if a container is alive — failure causes a restart. `readinessProbe` detects if a container is ready to serve traffic — failure removes it from Service endpoints without restarting it. Both run concurrently after the container starts.

Q14. What is the difference between a ConfigMap and a Secret?

> ConfigMap stores non-sensitive config as plain text. Secret stores sensitive data base64-encoded. Both can be used as environment variables or mounted as volumes. Secrets should use encryption at rest and be accessed via RBAC.

Q15. What is a PersistentVolume and a PersistentVolumeClaim?

> A PV is a piece of storage provisioned by an admin (or dynamically by a StorageClass). A PVC is a user's request for storage specifying size and access mode. K8s binds a PVC to a matching PV. This decouples storage provisioning from consumption.

Q16. What is a Service in Kubernetes? Explain the types.

> A Service provides a stable network endpoint to Pods. Types: `ClusterIP` (internal only), `NodePort` (exposes on each Node's IP at a static port), `LoadBalancer` (provisions a cloud LB), `ExternalName` (maps to an external DNS name).

Q17. How does Kubernetes service discovery work?

> K8s assigns each Service a DNS name in the format `<service>.<namespace>.svc.cluster.local`. CoreDNS resolves these names. Pods can reach any Service by its DNS name, and K8s updates the DNS records automatically.

Q18. What is a Headless Service?

> A Headless Service has `clusterIP: None` — it doesn't provide load balancing. DNS returns individual Pod IPs directly. Used with StatefulSets to address specific Pods (e.g., primary vs replica in a database cluster).

Q19. What is an Ingress?

> Ingress manages external HTTP/HTTPS access to cluster Services. It provides host-based routing, path-based routing, and TLS termination. It requires an Ingress Controller (like nginx-ingress) to function.

Q20. What is RBAC in Kubernetes?

> Role-Based Access Control controls who can do what to which resources. Key objects: `Role`/`ClusterRole` define permissions; `RoleBinding`/`ClusterRoleBinding` bind them to subjects (Users, Groups, ServiceAccounts).

Q21. What is a ResourceQuota?

> A ResourceQuota limits total resource consumption (CPU, memory, number of Pods/Services/PVCs) within a namespace, preventing one team from consuming all cluster resources.

Q22. What is a LimitRange?

> A LimitRange sets default resource requests and limits for containers in a namespace, and enforces minimum and maximum values. Ensures containers without explicit resources still have sensible defaults.

Q23. What is the QoS class of a Pod?

> QoS classes determine eviction priority. `Guaranteed` (requests=limits) is evicted last. `Burstable` (requests < limits) is middle priority. `BestEffort` (no resources set) is evicted first under pressure.

Q24. How do you perform a zero-downtime deployment in Kubernetes?

> Use a `Deployment` with `RollingUpdate` strategy. Set `maxUnavailable=0` (no Pods are taken down before new ones are ready) and `maxSurge=1` (one extra Pod created at a time). Also ensure `readinessProbe` is configured so traffic only goes to ready Pods.

Q25. What is a CronJob?

> A CronJob creates Jobs on a schedule (like cron). It supports cron syntax for scheduling, and settings like `concurrencyPolicy`, `successfulJobsHistoryLimit`, and `startingDeadlineSeconds`.

Advanced Level

Q26. Explain the Kubernetes control loop.

> Every controller in K8s runs a control loop: Watch the API server for resource changes → Compare actual state to desired state → Act to reconcile (create/update/delete resources) → Repeat. This is the fundamental reconciliation pattern.

Q27. What is a Custom Resource Definition (CRD)?

> A CRD extends the Kubernetes API with new resource types. Once created, you can use `kubectl` to manage custom resources like built-in ones. CRDs are the foundation of Kubernetes Operators.

Q28. What is a Kubernetes Operator?

> An Operator is a CRD + a custom controller that encodes operational knowledge about a specific application. It automates tasks like provisioning, scaling, backup, and recovery. Examples: postgres-operator, prometheus-operator.

Q29. What is Helm and what problem does it solve?

> Helm is the package manager for Kubernetes. It bundles K8s manifests into reusable Charts, supports templating (parameterised manifests), and manages release lifecycle (install, upgrade, rollback, uninstall).

Q30. What are Taints and Tolerations?

> Taints are applied to Nodes to repel Pods without a matching Toleration. Effects: `NoSchedule` (don't schedule), `PreferNoSchedule` (prefer not to), `NoExecute` (evict existing pods). Used to dedicate Nodes for specific workloads (GPU nodes, production-only nodes).

Q31. What is Pod Affinity and Anti-Affinity?

> Pod Affinity schedules Pods close to other Pods (same Node/zone). Pod Anti-Affinity spreads Pods away from each other (high availability across nodes/zones). Both support `required` (hard) and `preferred` (soft) rules.

Q32. How does the Horizontal Pod Autoscaler work?

> HPA queries the Metrics API every 15 seconds, calculates the desired replica count as `ceil(currentReplicas * (currentMetric / targetMetric))`, and scales the Deployment. Requires metrics-server for CPU/memory metrics.

Q33. What is a Network Policy?

> A NetworkPolicy is a firewall rule at the Pod level. It controls ingress and egress traffic using Pod/Namespace selectors. By default, all traffic is allowed — NetworkPolicies restrict it. Requires a CNI plugin that supports NetworkPolicies.

Q34. How would you debug a Pod that's stuck in CrashLoopBackOff?

> 1. `kubectl describe pod <pod>` — check Events for error messages. 2. `kubectl logs <pod> --previous` — check logs from crashed container. 3. Check resource limits (OOMKilled?). 4. Check liveness probe configuration. 5. `kubectl exec` to shell in temporarily. 6. Change command to `sleep infinity` to keep container alive for inspection.

Q35. What is the difference between emptyDir and hostPath volumes?

> `emptyDir` is a temporary volume tied to the Pod lifecycle — deleted when Pod is removed, shared between containers in the Pod. `hostPath` mounts a Node's filesystem — persists beyond Pod lifetime but ties the Pod to a specific Node and has security implications.

Q36. How does rolling back a Deployment work?

> Each Deployment update creates a new ReplicaSet while keeping the old one (with 0 replicas). `kubectl rollout undo` scales up the previous ReplicaSet and scales down the

> current one. History depth is controlled by `revisionHistoryLimit`.

Q37. What is the init container and when would you use it?

> Init containers run sequentially before main containers start. They must complete successfully for the main containers to start. Use cases: wait for a dependency (DB, config service), run migrations, set up permissions, populate shared volumes.

Q38. How does CoreDNS work in Kubernetes?

> CoreDNS is the cluster DNS server running as a Deployment in `kube-system`. Every Pod is configured to use it as its DNS resolver. It responds to `<service>.<namespace>.svc.cluster.local` queries, returning the Service's ClusterIP.

Q39. What is a ServiceAccount and how is it used?

> A ServiceAccount provides an identity for Pods running in a cluster. It's used for authentication when Pods need to call the Kubernetes API or access other services. Tokens are automatically mounted at `/var/run/secrets/kubernetes.io/serviceaccount/`.

Q40. What is the difference between Deployment, StatefulSet, DaemonSet, and Job?

> `Deployment`: stateless apps, any number of replicas, random Pod names. `StatefulSet`: stateful apps, ordered Pods with stable IDs and individual PVCs. `DaemonSet`: one Pod per Node, for agents/daemons. `Job`: runs Pods to completion (batch tasks). `CronJob`: scheduled Jobs.

Q41. How would you securely pass secrets to a Pod?

> 1. Create a K8s Secret with the sensitive data. 2. Mount it as an environment variable or volume in the Pod spec. 3. Enable etcd encryption at rest. 4. Use RBAC to restrict who can read Secrets. 5. Consider external secret managers (HashiCorp Vault, AWS Secrets Manager) with integration tools like External Secrets Operator.

Q42. What is Pod Disruption Budget (PDB)?

> A PDB limits the number of Pods of a replicated application that can be simultaneously disrupted during voluntary disruptions (node drain, cluster upgrade). Prevents taking down too many replicas at once.

```yaml
```

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2         # or maxUnavailable: 1
  selector:
    matchLabels:
      app: my-app
```

Q43. What is a sidecar container pattern?

> A sidecar is a secondary container in the same Pod that extends or enhances the main container. Examples: log shipper (sends main app logs to Elasticsearch), service mesh proxy (Envoy/Istio), metrics exporter, security scanner.

Q44. What is Kubernetes Federation?

> Federation enables managing multiple Kubernetes clusters as a single entity. It allows deploying workloads across clusters for disaster recovery, geo-distribution, and separation of concerns.

Q45. What is a Topology Spread Constraint?

> It controls how Pods are spread across the cluster topology (zones, nodes, regions). Ensures high availability by preventing Pods from being concentrated in one zone or node.

Q46. How do you upgrade a Kubernetes cluster?

> 1. Backup etcd. 2. Upgrade control plane (one node at a time for HA). 3. Upgrade kubelet and kubectl on control plane nodes. 4. Upgrade worker nodes (drain → upgrade → uncordon). 5. Use `kubeadm upgrade` for kubeadm-managed clusters.

Q47. What is CNI and name some implementations?

> CNI (Container Network Interface) is the plugin interface for Pod networking. Implementations: Calico (network policy + routing), Flannel (simple overlay network), Cilium (eBPF-based, advanced policies), Weave Net, AWS VPC CNI, Azure CNI.

Q48. What is the difference between port, targetPort, and nodePort in a Service?

> `port` is the port the Service listens on within the cluster. `targetPort` is the port the container application listens on. `nodePort` is the external port exposed on every Node (only for NodePort Services).

Q49. How does Kubernetes achieve high availability for the control plane?

> Multiple control plane nodes run `kube-apiserver`, `kube-controller-manager`, and `kube-scheduler`. A load balancer distributes traffic to API servers. etcd runs as a cluster (odd number: 3 or 5 nodes). Leader election is used for controller-manager and scheduler.

Q50. What is Vertical Pod Autoscaler (VPA)?

> VPA automatically adjusts CPU and memory requests/limits for containers based on historical usage. Unlike HPA (which changes replica count), VPA changes the resource allocation per Pod. When updating, it must restart Pods to apply new resources.

Expert / Scenario Level

Q51. How would you handle an OOMKilled Pod?

> 1. `kubectl describe pod` to confirm OOMKilled. 2. `kubectl top pod` and check actual memory usage vs limits. 3. Increase `resources.limits.memory`. 4. Investigate memory leak in application. 5. Enable VPA to auto-tune. 6. Consider breaking the app into smaller microservices.

Q52. How would you implement blue-green deployment in Kubernetes?

> Run two Deployments (blue=current, green=new). Both have separate label sets. The Service selector points to blue. Deploy and test green. Switch Service selector to green. Both versions run simultaneously — instant switchover. Roll back by switching selector back to blue.

```yaml
# Switch service to green
kubectl patch service my-app -p '{"spec":{"selector":{"version":"green"}}}'
```

Q53. How would you implement canary deployment?

> Run two Deployments with the same label (`app: my-app`) but different `version` labels. The Service selects on `app: my-app` (both). Control traffic split by controlling replica ratio. With 9 stable + 1 canary Pod = 10% canary traffic. Use Ingress annotations or service mesh for percentage-based routing.

Q54. What are Finalizers?

> Finalizers are keys placed on objects to signal pre-deletion operations must complete before the object is deleted. When you delete an object with finalizers, K8s sets `deletionTimestamp` and waits. A controller performs cleanup and removes the finalizer. Only then is the object actually deleted.

Q55. How would you design a multi-tenant Kubernetes cluster?

> 1. Use Namespaces per tenant. 2. ResourceQuotas to limit consumption per namespace. 3. LimitRanges for default container limits. 4. RBAC to restrict access to each tenant's namespace. 5. NetworkPolicies to isolate Pod communication. 6. Separate node pools with taints/tolerations for strong isolation. 7. Consider dedicated clusters for highest isolation requirements.

Q56. What is Istio and what problems does it solve?

> Istio is a service mesh that injects Envoy proxies (sidecars) into every Pod. It provides: mutual TLS (mTLS) for service-to-service encryption, traffic management (circuit breaking, retries, canary routing), observability (distributed tracing, metrics), and fine-grained access policies — all without modifying application code.

Q57. What is the difference between imperative and declarative management?

> Imperative: `kubectl create`, `kubectl run`, `kubectl scale` — you specify what to DO. Declarative: `kubectl apply -f manifest.yaml` — you specify what you WANT (desired state), K8s figures out how to get there. Declarative is preferred for production — supports version control, repeatability, and GitOps.

Q58. How does Kubernetes handle secrets securely in production?

> 1. Enable etcd encryption at rest. 2. Use strict RBAC (least privilege). 3. Mount secrets as volumes (not env vars — easier to audit). 4. Integrate external secret managers (Vault, AWS Secrets Manager). 5. Use External Secrets Operator to sync external secrets into K8s. 6. Rotate secrets regularly. 7. Audit secret access in audit logs.

Q59. What is GitOps and how does it relate to Kubernetes?

> GitOps is a deployment paradigm where Git is the single source of truth. All K8s manifests are in Git. A GitOps agent (ArgoCD, Flux) watches the Git repo and automatically syncs changes to the cluster. Benefits: audit trail, easy rollback, consistency, automated drift detection.

Q60. How would you optimise Kubernetes costs in a cloud environment?

> 1. Right-size resources using VPA recommendations. 2. Use HPA to scale down during off-peak hours. 3. Use Cluster Autoscaler to remove unused nodes. 4. Schedule batch jobs on Spot/Preemptible instances. 5. Use `PodDisruptionBudget` with Spot nodes. 6. Set resource quotas to prevent waste. 7. Use namespace-level resource monitoring. 8. Pack Pods efficiently with bin-packing node affinity. 9. Delete unused PVCs and LoadBalancers.

---

25. Quick Reference Cheat Sheet

Object Abbreviations

```bash
po      pods
rs      replicasets
deploy  deployments
svc     services
ns      namespaces
cm      configmaps
sa      serviceaccounts
pv      persistentvolumes
pvc     persistentvolumeclaims
hpa     horizontalpodautoscalers
ing     ingresses
ds      daemonsets
sts     statefulsets
job     jobs
cj      cronjobs
no      nodes
ep      endpoints
```

Must-Know YAML Fields

```yaml
apiVersion:   # API version (v1, apps/v1, batch/v1)
kind:         # Object type (Pod, Deployment, Service, etc.)
metadata:
  name:       # Object name
  namespace:  # Namespace (default if omitted)
  labels:     # Key-value tags for selection
  annotations:# Non-identifying metadata
spec:         # Desired state (differs per object type)
status:       # Current state (managed by K8s — do not edit)
```

Resource Capacity Units

```
CPU:     1 = 1 core, 1000m = 1 core, 500m = 0.5 core
Memory:  Ki, Mi, Gi, Ti (binary) | K, M, G, T (decimal)
```

```
Storage: Same as memory units
```

Pod Restart Policies

```
Always     → always restart (default, for Deployments)
OnFailure  → restart only if non-zero exit code (for Jobs)
Never      → never restart (for one-shot tasks)
```

Service Port Ranges

```
NodePort:  30000-32767 (configurable)
ClusterIP: auto-assigned from service-cluster-ip-range
```

Common kubectl Patterns

```bash
# Get resource YAML
kubectl get <resource> <name> -o yaml

# Watch for changes
kubectl get pods -w

# Dry run (validate without applying)
kubectl apply -f manifest.yaml --dry-run=client
kubectl apply -f manifest.yaml --dry-run=server

# Force replace
kubectl replace --force -f manifest.yaml

# Get events for a pod
kubectl get events --field-selector involvedObject.name=my-pod

# Check API resources
kubectl api-resources

# Explain a field
kubectl explain pod.spec.containers.resources
kubectl explain deployment.spec.strategy
```

Debugging Checklist

```
Pod not starting?
  ✓ kubectl describe pod → check Events
  ✓ kubectl logs --previous → check crash logs
  ✓ Image pull error → check imagePullSecrets, image name/tag
  ✓ OOMKilled → increase memory limits
  ✓ CrashLoopBackOff → fix app crash or probe config

Pod not receiving traffic?
  ✓ kubectl get endpoints → check if pod IPs are listed
  ✓ Labels on pod match service selector?
  ✓ readinessProbe passing?
  ✓ Port in service matches containerPort?

Node not ready?
  ✓ kubectl describe node → check Conditions
  ✓ kubectl get events → look for disk/memory pressure
  ✓ systemctl status kubelet → kubelet health

Ingress not working?
  ✓ Ingress controller pods running?
  ✓ Correct ingressClassName set?
  ✓ Service behind ingress healthy?
  ✓ DNS pointing to ingress controller's external IP?
```

🔧 Setup & Local Development

Install kubectl

```bash
# macOS
brew install kubectl

# Linux
curl -LO https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)/
chmod +x kubectl && mv kubectl /usr/local/bin/

# Windows (Chocolatey)
choco install kubernetes-cli
```

Local Kubernetes Options

Tool	Description	Best for
Minikube	Single-node cluster	Beginners, local dev
kind (K8s in Docker)	Multi-node in Docker containers	CI, testing
k3s	Lightweight K8s	Resource-constrained, IoT, RPi
Docker Desktop	Built-in K8s	Mac/Windows developers
Rancher Desktop	Free Docker Desktop alternative	Mac/Windows developers

```bash
# Minikube quick start
brew install minikube
minikube start --cpus=4 --memory=8g
minikube dashboard
minikube tunnel   # expose LoadBalancer services locally

# kind quick start
brew install kind
kind create cluster --name my-cluster
kind create cluster --config multi-node.yaml
```

Study tip: Read each section, understand the YAML structure, then practice running each example in a real cluster (Minikube or kind). The interview questions mirror exactly what is covered in these notes. ☸